

Gaussian random fields and processes

Gaussian processes

We have seen Gaussian random variables and their (finite-dimensional) multivariate form.

This can be further generalised to Gaussian random fields (or processes, both mean the same thing), which are (infinite-dimensional) distributions over functions.

In a Gaussian process, at any finite set of points $x_1, \dots, x_n$, the values $y_1, \dots, y_n$ are jointly Gaussian distributed:

$$(y_1, \dots, y_n) \sim \mathcal{N}((\mu_{x_1}, \dots, \mu_{x_n}), \Sigma_{x_1, \dots, x_n}) .$$

As a whole, a Gaussian process is a distribution over *functions* $f(x) = y$:

$$f \sim \mathcal{GP}(\mu, \Sigma) ,$$

where $\mu(x)$ and $\Sigma(x, x')$ are functions of the positions x .

This is a very flexible and powerful way to specify functions.

Because of its probabilistic nature, it allows sampling and getting uncertainty estimates as well.

Helpful resources on Gaussian processes:

- The standard reference book: <http://gaussianprocess.org/gpml/chapters/>
- In-depth explanation and discussion of some of the possible pitfalls: https://betanalpha.github.io/assets/case_studies/gaussian_processes.html
- Nice intuitive explanation and visualisations: <https://distill.pub/2019/visual-exploration-gaussian-processes/>

Gaussian processes can come in handy when we want to model data for which we do not have a physical model.

For example, poorly understood observational or measurement effects.

In this case, we might have a mean function $\mu(x)$ that provides a physical model for the contributions to the data we understand (and care about), while we use a Gaussian process to model the poorly-understood residuals.

For this reason, Gaussian processes often assume that the mean is zero, since we can always subtract a mean function from the data:

$$f_{\text{residual}} \sim \mathcal{GP}(0, \Sigma) \quad \leftrightarrow \quad f_{\text{residual}} + \mu \sim \mathcal{GP}(\mu, \Sigma)$$

The data is often homogeneous (translation-invariant, stationary), that is, the covariance does not depend on the absolute position but only on their differences:

$$\Sigma(x, x') = \Sigma(x - x').$$

If the data is furthermore isotropic (rotation-invariant), the covariance only depends on the norm of the difference in the positions:

$$\Sigma(x, x') = \Sigma(|x - x'|).$$

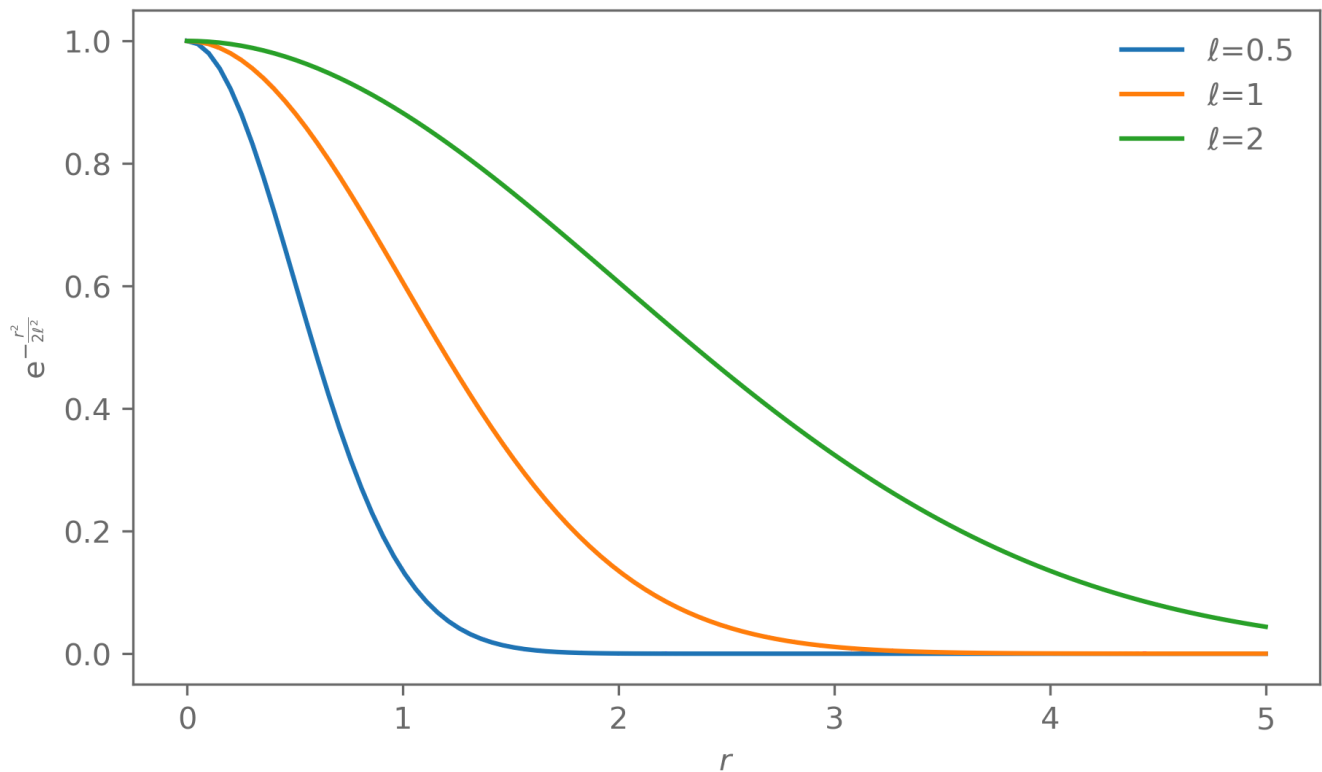
In the context of Gaussian processes, the covariance function is also called the *kernel* $k(x, x') = \Sigma(x, x')$.

This is the main modelling choice of a Gaussian process.

A common choice is the exponential squared or radial basis function kernel:

$$k(x, x') = e^{-\frac{1}{2} \frac{\|x - x'\|^2}{\ell^2}}$$

The parameter of the kernel is its length-scale ℓ : larger ℓ mean points further apart are more correlated.



Kernels can be added and multiplied, for more flexible modelling of the correlation structure.

For example, scaling the exponential squared kernel allows us to change the overall variance of the distribution over functions:

$$k(x, x') = \sigma^2 e^{-\frac{1}{2} \frac{\|x - x'\|^2}{\ell^2}}$$

To implement the Gaussian process, we use the `tinygp` package:

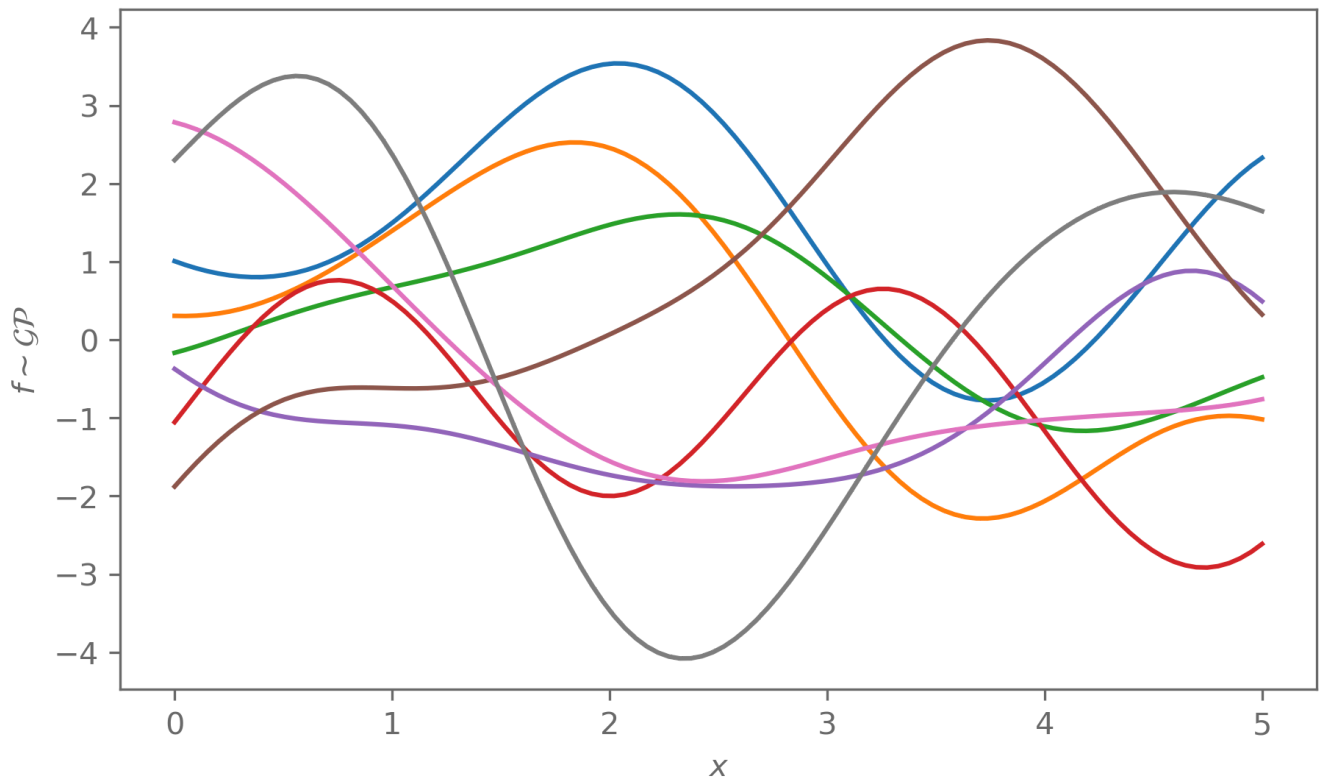
```
import tinygp

random_key, main_key = jax.random.split(jax.random.PRNGKey(42))

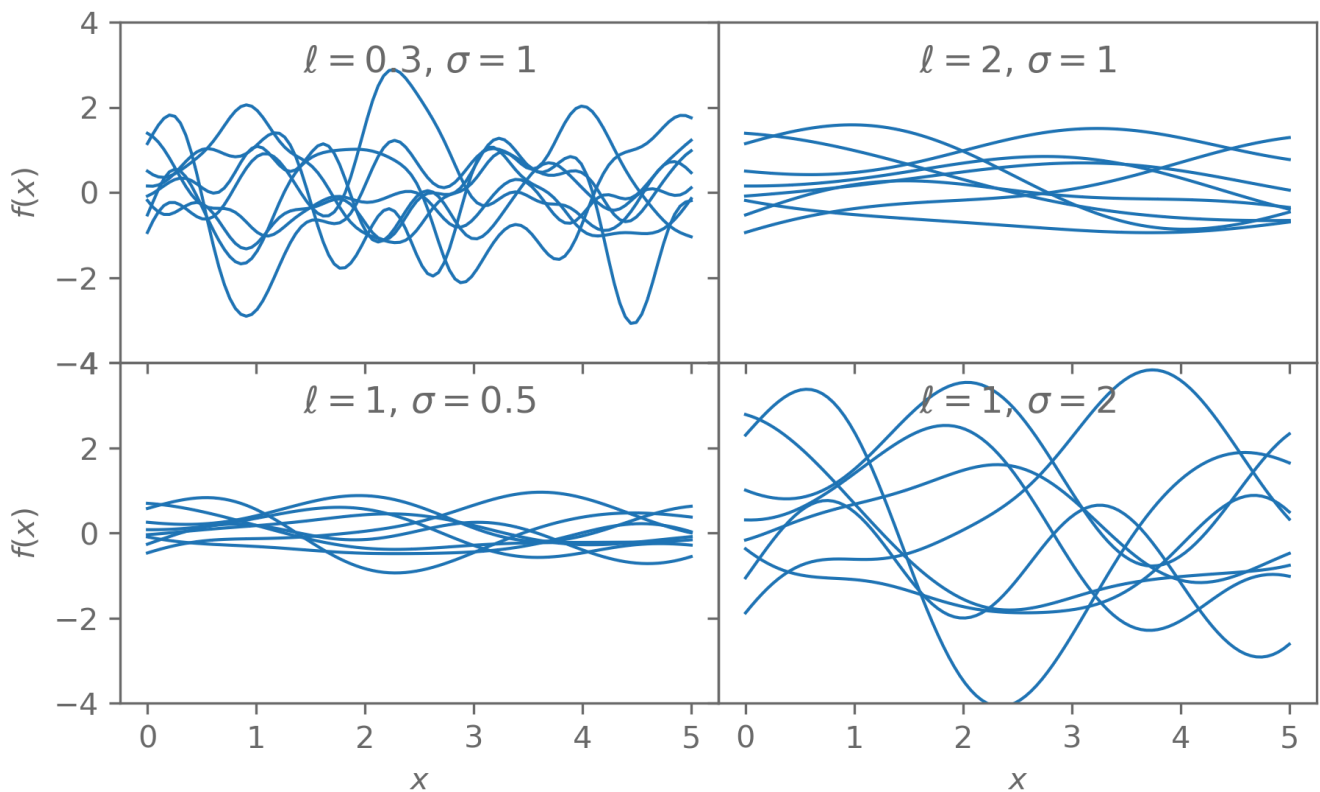
# A exponential squared kernel with length scale 2 and scaled by
# an amplitude 1.3^2
kernel = 1.3**2 * tinygp.kernels.ExpSquared(scale=2.0)

x = jnp.linspace(0, 5, 100)
# The Gaussian process is evaluated at a set of points x and
# depends on the kernel.
gp = tinygp.GaussianProcess(kernel, x)

# We can now draw function samples (evaluated at x) from the GP
gp_prior_samples = gp.sample(key=random_key, shape=(8,))
```



Different length scales and amplitudes affect the distribution of functions the Gaussian process samples:



What is actually happening here under the hood?

The kernel $k(x, x')$ specifies the covariance between input positions x and x' .

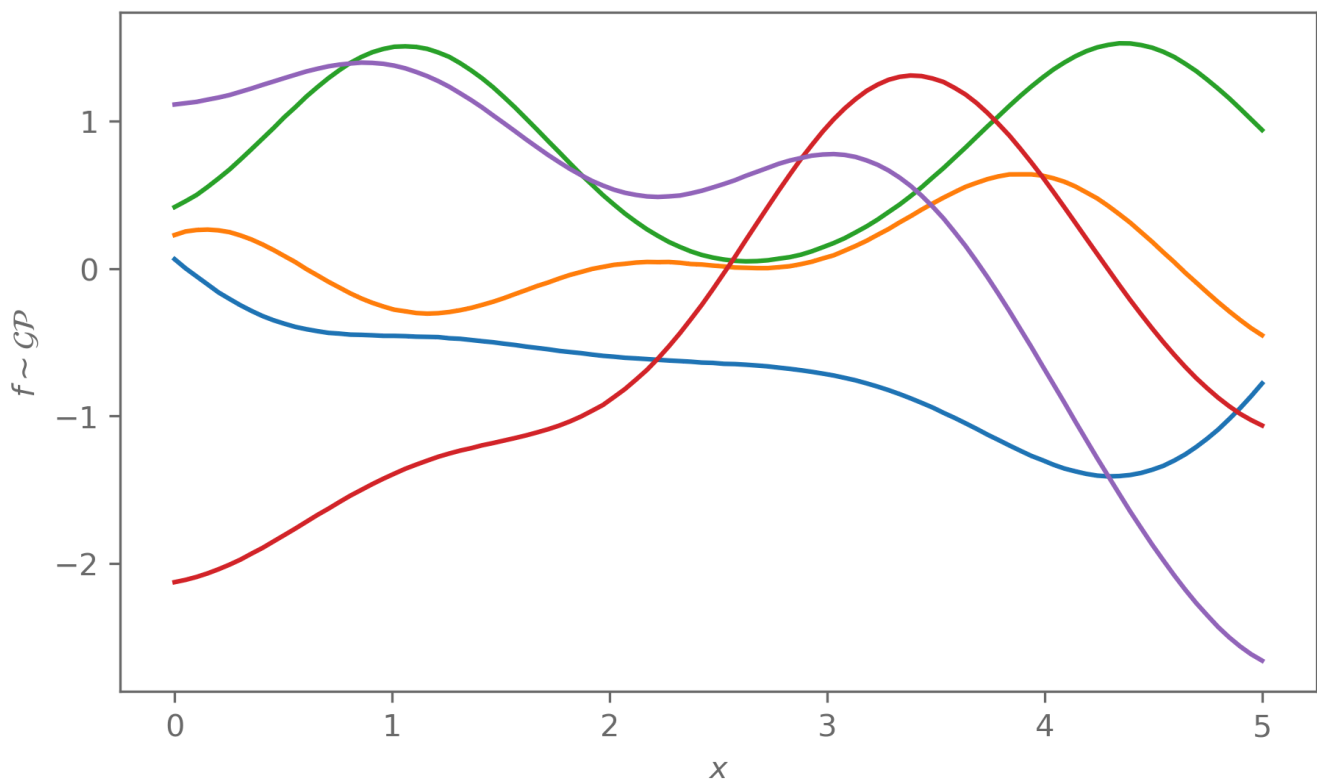
To draw a sample function f , evaluated at some points x_i , we draw from a multivariate Gaussian distribution with covariance matrix

$$\Sigma_{ij} = k(x_i, x_j).$$

```
kernel = tinygp.kernels.ExpSquared(scale=1.0)
x = jnp.linspace(0, 5, 100)
k_matrix = kernel(x, x)

# Need to add a small component to the diagonal to make it numerically stable
k_matrix += 1e-6*jnp.eye(x.size)

# Draw correlated samples by multiplying iid standard normals with the
# Cholesky decomposition of the covariance matrix
L = jnp.linalg.cholesky(k_matrix)
samples = L @ jax.random.normal(key=random_key, shape=(x.size, 5))
```



So far we only used the Gaussian process to draw samples of functions, without relating it to any specific data.

In other words, these were samples from some prior over functions.

One key advantage of Gaussian processes is that we can easily condition them on observations.

Given a set of observed function values $f_i = f(x_i)$ at positions x_i , we can get the distribution of function values f_j^* at some other set of positions x_j^* .

We first note that the joint distribution of $\vec{f} = (f_1, \dots, f_n)^T$ and $\vec{f}^* = (f_1^*, \dots, f_n^*)^T$ is again a multivariate Gaussian:

$$\begin{pmatrix} \vec{f} \\ \vec{f}^* \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \vec{\mu} \\ \vec{\mu}^* \end{pmatrix}, \begin{pmatrix} \Sigma_{xx} & \Sigma_{xx^*} \\ \Sigma_{xx^*}^T & \Sigma_{x^*x^*} \end{pmatrix} \right).$$

The means and covariances are those of the Gaussian process:

$$\begin{aligned} \vec{\mu} &= (\mu(x_1), \dots, \mu(x_n))^T \\ \vec{\mu}^* &= (\mu(x_1^*), \dots, \mu(x_n^*))^T \\ \Sigma_{xx} &= k(\vec{x}, \vec{x}) \\ \Sigma_{xx^*} &= k(\vec{x}, \vec{x}^*) \\ \Sigma_{x^*x^*} &= k(\vec{x}^*, \vec{x}^*) \end{aligned}$$

Because we are dealing with Gaussian distributions, we can use the fact that there is an exact expression for their conditional distributions:

$$\begin{aligned} \vec{f}^* | \vec{f} &\sim \mathcal{N} \left(\vec{\mu}_{x^*|x}, \Sigma_{x^*|x} \right) \\ \vec{\mu}_{x^*|x} &= \vec{\mu}^* + \Sigma_{x^*x} \Sigma_{xx}^{-1} (\vec{f} - \vec{\mu}) \\ \Sigma_{x^*|x} &= \Sigma_{x^*x^*} - \Sigma_{x^*x} \Sigma_{xx}^{-1} \Sigma_{xx^*} \end{aligned}$$

This ability to make predictions at new positions based on observations becomes especially useful when the likelihood is Gaussian.

Let us say that our data y_i at positions x_i have a Gaussian likelihood with mean $f(x_i)$ and variance σ_y^2 :

$$y_i \sim \mathcal{N}(f(x_i), \sigma_y^2).$$

If f is a Gaussian process with mean μ and kernel k , $f \sim \mathcal{GP}(\mu, k)$, then

$$y_i \sim \mathcal{N}(\mu(x_i), \Sigma_{x_i x_i})$$

with

$$\Sigma_{x_i x_j} = k(x_i, x_j) + \delta_{ij} \sigma_y^2.$$

For a Gaussian likelihood, the posterior over functions f^* is again a Gaussian process and we can use the same conditioning machinery:

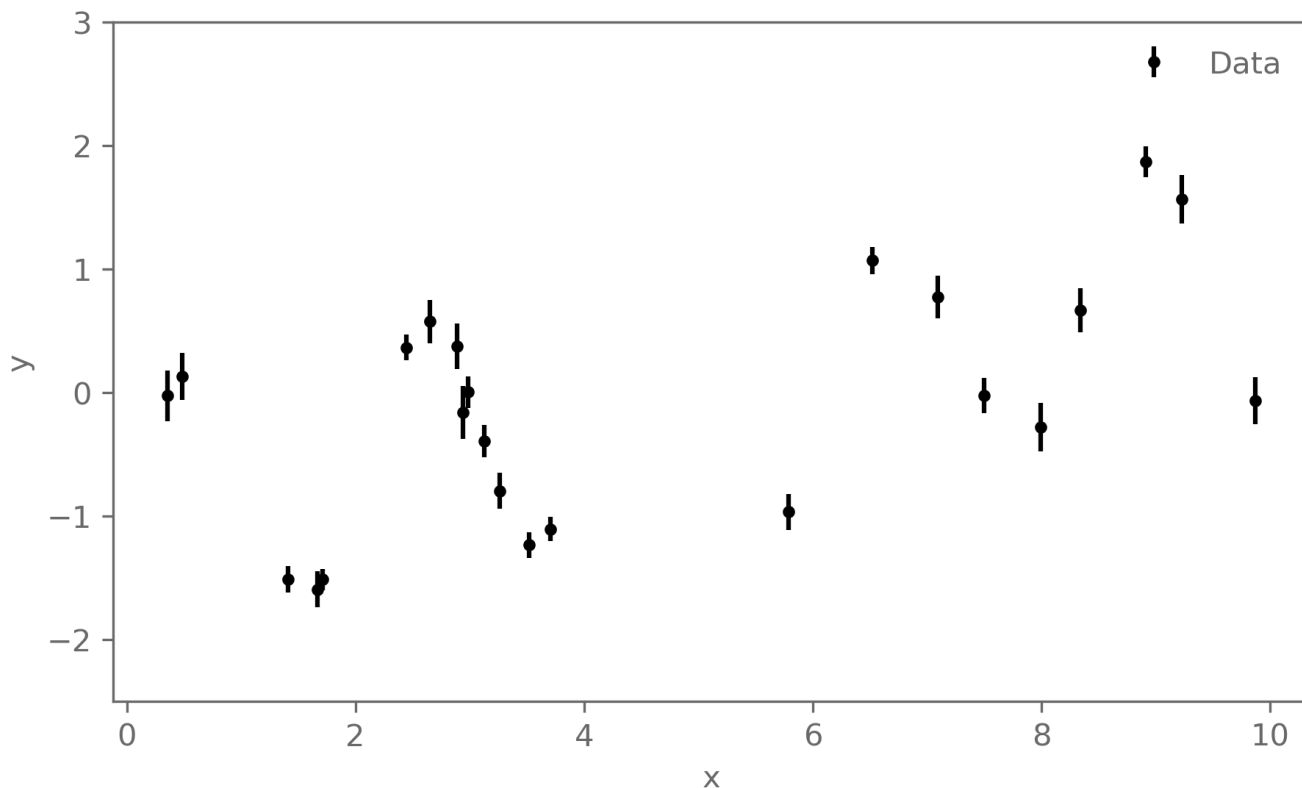
$$\vec{f}^* | \vec{y} \sim \mathcal{N} \left(\vec{\mu}_{x^*|x}, \Sigma_{x^*|x} \right) .$$

The covariance at the observations Σ_{xx} now includes the observational uncertainty σ_y on the diagonal:

$$\Sigma_{x_i x_j} = k(x_i, x_j) + \delta_{ij} \sigma_y^2 .$$

The diagonal observational uncertainty $\delta_{ij} \sigma_y$ can be generalised to a full noise matrix Σ_{ij}^y .

Let us see how this works on some synthetic data:



We are given observations `y_obs` with uncertainties `y_err` at positions `x_obs` .

To start, we try a simple exponential square kernel and set up the GP in `tinygp` .

```
l = 1.5
sigma = 1

kernel = sigma**2 * tinygp.kernels.ExpSquared(scale=l)

gp = tinygp.GaussianProcess(
    kernel, x_obs, diag=y_err**2,
)
```

Now comes the key part: we condition the GP on the observed data `y_obs` and make predictions at some other positions.

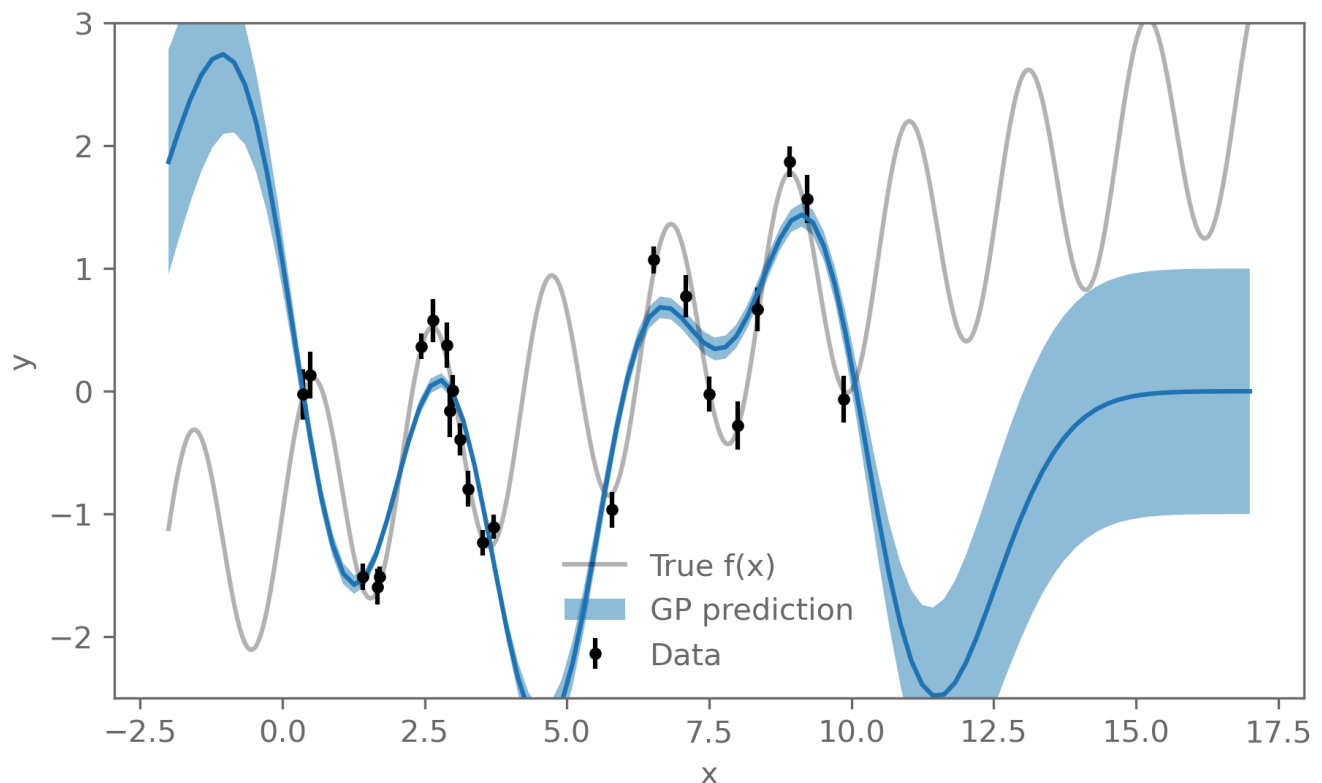
```

# Define the points where we want the GP to make prediction
x_pred = np.linspace(-2, 17, 100)

# Now we can condition the GP on the observed values y_obs and get predictions
# at x_pred. This is again a GP
conditioned_gp = gp.condition(y=y_obs, X_test=x_pred).gp

# Extract the mean and variance of the prediction (we get the uncertainty)
# for free!
mu, std = conditioned_gp.loc, jnp.sqrt(conditioned_gp.variance)

```



Not terrible, not great.

We can evaluate the probability of the observed data `y_obs` under the GP model:

```
gp.log_probability(y=y_obs)
```

```
Array(-122.66108481, dtype=float64)
```

By maximising this probability (the likelihood), we can find values for the parameters of the kernel that describe the data better.

```

def build_gp(params, x_obs, y_err):
    l = jnp.exp(params["log_l"])
    sigma = jnp.exp(params["log_sigma"])
    kernel = sigma**2 * tinygp.kernels.ExpSquared(scale=l)

    gp = tinygp.GaussianProcess(
        kernel, x_obs, diag=y_err**2,

```

```

    )
    return gp

# For the minimisation, we need the negative log likelihood
def neg_log_likelihood(params, x_obs, y_obs, y_err):
    gp = build_gp(params, x_obs, y_err)
    return -gp.log_probability(y=y_obs)

```

```

params_init = dict(
    log_l = jnp.log(1),
    log_sigma = jnp.log(1)
)
print(f"Initial negative log likelihood: "
      f"{neg_log_likelihood(params_init, x_obs, y_obs, y_err)}")

# Find the maximum of the likelihood
solver = jaxopt.ScipyMinimize(fun=neg_log_likelihood)
result = solver.run(params_init, x_obs=x_obs, y_obs=y_obs, y_err=y_err)

print(f"Final negative log likelihood: {result.state.fun_val}")
print(f"Parameter values: {result.params}")

```

```

Initial negative log likelihood: 37.63546421761678
Final negative log likelihood: 12.760563475310185
Parameter values: {'log_l': Array(-0.50627298, dtype=float64), 'log_sigma': Array(0.0
405664, dtype=float64)}

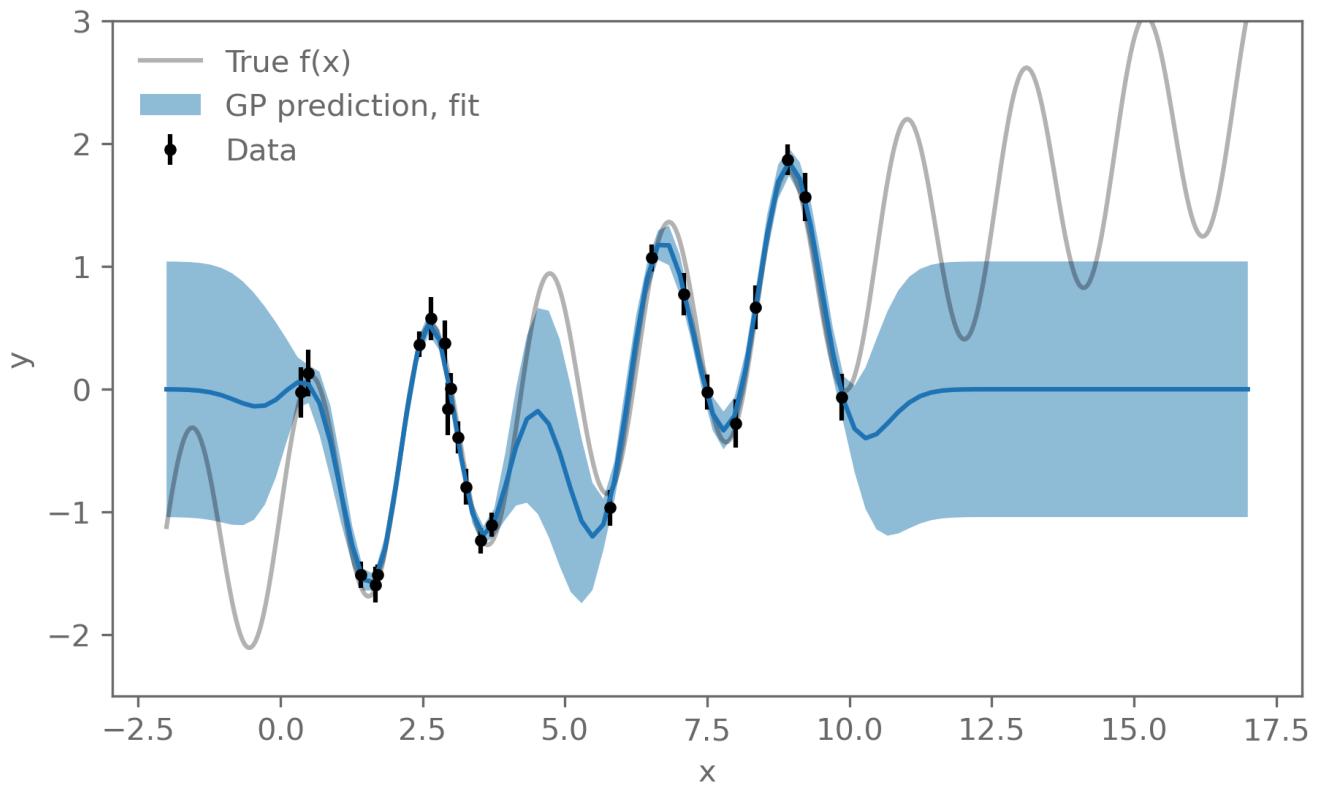
```

```

# Build the GP with the fitted parameters
gp = build_gp(result.params, x_obs=x_obs, y_err=y_err)

conditioned_gp = gp.condition(y=y_obs, X_test=x_pred).gp
mu, std = conditioned_gp.loc, jnp.sqrt(conditioned_gp.variance)

```



Better, but the extrapolation is still poor.

To do better, we could try a different kernel.

Here we try a kernel that can account for a periodic signal.

We also add a simple linear model for the mean.

```
def build_gp(params, x_obs, y_err):
    period = params["period"]
    gamma = jnp.exp(params["log_gamma"])
    sigma = jnp.exp(params["log_sigma"])
    slope = params["slope"]
    kernel = sigma**2 * tinygp.kernels.ExpSineSquared(scale=period, gamma=gamma)

    mean_offset = params["mean"]
    mean_func = lambda x: mean_offset + slope*x

    gp = tinygp.GaussianProcess(
        kernel, x_obs, diag=y_err**2, mean=mean_func
    )
    return gp
```

```
def neg_log_likelihood(params, x_obs, y_obs, y_err):
    gp = build_gp(params, x_obs, y_err)
    return -gp.log_probability(y=y_obs)

params_init = dict(
    period = 6.0,
```

```

log_gamma = jnp.log(1),
log_sigma = jnp.log(1),
slope=0.0,
mean=0.0
)

print(f"Initial negative log likelihood: {neg_log_likelihood(params_init, x_obs=x_obs,
solver = jaxopt.ScipyMinimize(fun=neg_log_likelihood)
soln = solver.run(params_init, x_obs=x_obs, y_obs=y_obs, y_err=y_err)
print(f"Final negative log likelihood: {soln.state.fun_val}")
soln.params

```

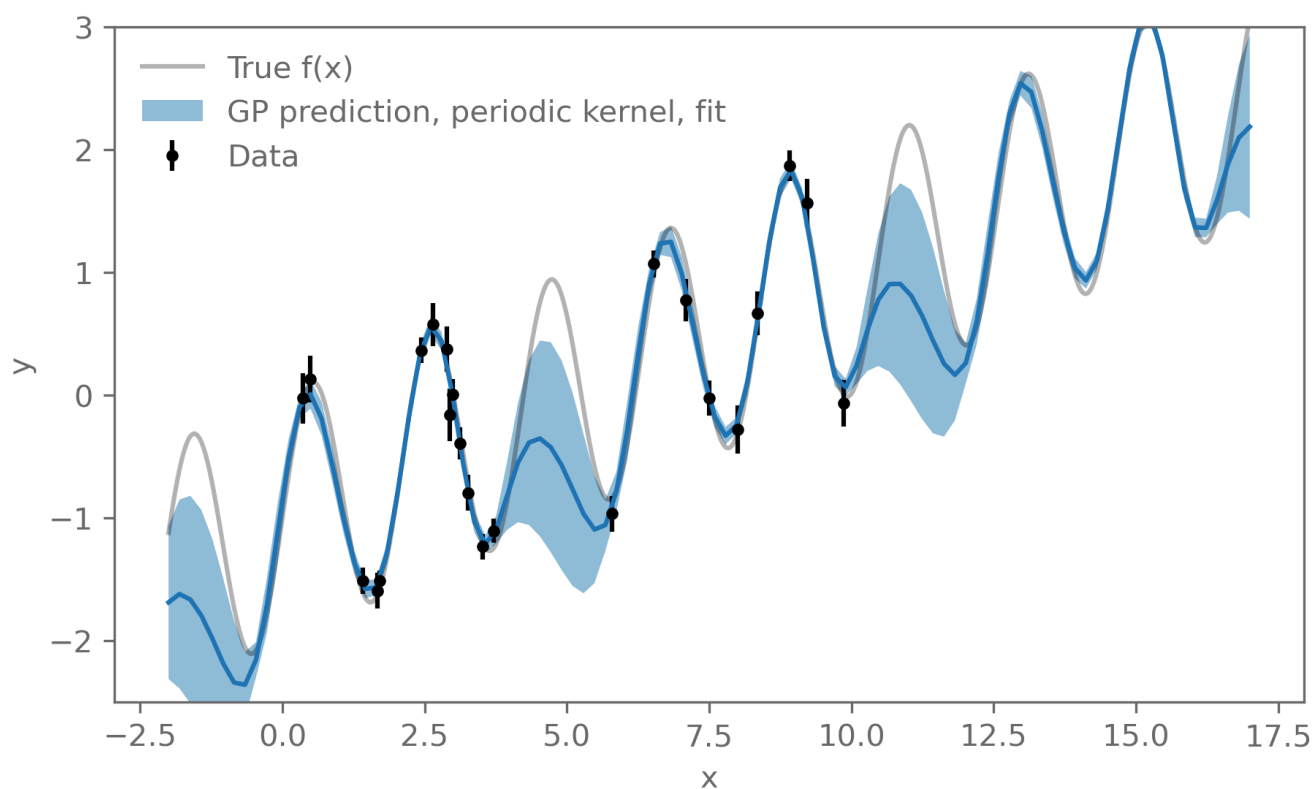
Initial negative log likelihood: 280.8865902954099

Final negative log likelihood: 2.000366566284147

```

{'log_gamma': Array(1.8516156, dtype=float64),
 'log_sigma': Array(-0.0733643, dtype=float64),
 'mean': Array(-1.23548533, dtype=float64),
 'period': Array(6.28013976, dtype=float64),
 'slope': Array(0.2021927, dtype=float64)}

```



Caveats to keep in mind

Evaluating the Gaussian process likelihood and conditioning both depend on the inverse of the covariance matrix at the position of the observations Σ_{xx} .

For n observations, this is a $n \times n$ matrix, which takes $\mathcal{O}(n^3)$ operations to invert. For large n , this becomes slow quickly.

Finding approximate methods that get around this n^3 scaling is a big field of research.

Because Gaussian process models are so flexible, we need to be careful with parameter degeneracies.

https://betanalpha.github.io/assets/case_studies/gaussian_processes.html#3_Inferring_A_Gaussian_Proc has a very insightful description of a pathological (but not unrealistic) example and how to address it.

The take-away message is that the data need to be able to constrain the parameters of the Gaussian process, otherwise issues can arise. For example, the data can only inform the length scale in a kernel for scales larger than the smallest separation between data points and on scales smaller than the largest separation.

Use of appropriate priors can avoid the optimisation or sampling algorithms getting stuck in degenerate parts of the parameter space.

Probabilistic programming

The proper way is of course to sample the parameters of the kernel, instead of just finding the best-fit values by maximising the likelihood.

We will do that using `numpyro`, a probabilistic programming framework built on `JAX`.

In probabilistic programming, variables our programs are random variable. We specify their distributions and the probabilistic programming framework takes care of propagating the distributions.

This will be a very brief introduction. For a slightly more detailed introduction, have a look at <https://dfm.io/posts/intro-to-numpyro/>.

To illustrate how this works, we first revisit the linear-fit data set with homoscedastic errors in `data/linear_fits/data_0.txt`.

We assume a Gaussian likelihood with $\mu(x) = mx + b$ and uncertainty σ_y :

$$y_i \sim \mathcal{N}(\mu(x_i), \sigma_y^2).$$

The parameters m and b both have Gaussian priors:

$$p(m) = \mathcal{N}(1, 1)$$

$$p(b) = \mathcal{N}(0, 1.2^2)$$

Our probabilistic model is then

$$m \sim \mathcal{N}(1, 1) \tag{1}$$

$$b \sim \mathcal{N}(0, 1.2^2) \tag{2}$$

$$y_i \sim \mathcal{N}(mx_i + b, \sigma_y^2) \tag{3}$$

In probabilistic programming frameworks we can specify such a model and then condition it on the actual observed data y_i^{obs} .

Instead of writing down (log) probability functions for the priors and likelihood to sample from the posterior, we can write down the model just in terms of its random variables, their distributions, and how they are related, and the probabilistic programming framework takes care of tracking all the probabilities.

```
x_obs, y_obs, y_err = np.loadtxt("data/linear_fits/data_0.txt", unpack=True)
# For now, all errors are the same
sigma_y = y_err[0]

def plot_linear_data_set(x, y, y_err=None):
    """Plot linear data set and format the plot."""
    fig, ax = plt.subplots(1, 1)

    if y_err is not None:
        ax.errorbar(x, y, y_err, fmt=".", label="Data")
    else:
        ax.plot(x, y, marker=".", linestyle="none", label="Data")

    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")
    ax.legend(loc="upper left")

    return fig, ax
```

```
import numpyro
import numpyro.distributions as distr

# Our probabilistic model needs to be defined as a function
def model(y_obs=None):
    # The parameters are drawn from the prior distributions
    m = numpyro.sample("m", distr.Normal(loc=1, scale=1))
    b = numpyro.sample("b", distr.Normal(loc=0, scale=1.2))

    # We save the mean predictions using the numpyro.deterministic primitive
    mu = numpyro.deterministic("mu", m*x_obs + b)

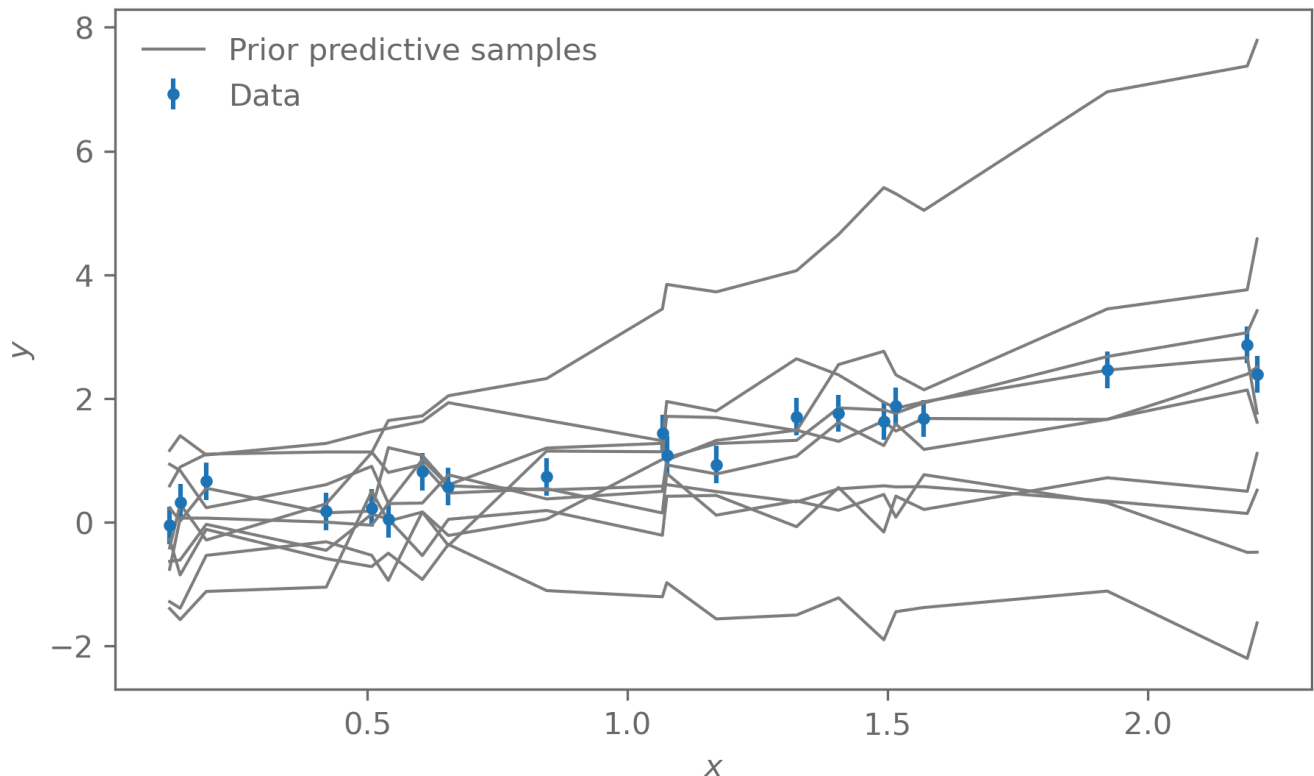
    # At the end we draw the observations and condition them on the actual
```

```
# observed data
numpyro.sample("obs", distr.Normal(loc=mu, scale=sigma_y), obs=y_obs)
```

Once we have the model specified, we can create samples from the prior predictive distribution.

```
master_key = jax.random.PRNGKey(42)
rng_key, master_key = jax.random.split(master_key)

prior_predictive = numpyro.infer.Predictive(model=model, num_samples=10)
prior_predictive_samples = prior_predictive(rng_key=rng_key)["obs"]
```



To get the posterior, we can use the build in NUTS HMC sampler.

```
rng_key, master_key = jax.random.split(master_key)

# Run NUTS
kernel = numpyro.infer.NUTS(model)
num_samples = 2000
mcmc = numpyro.infer.MCMC(kernel, num_warmup=1000, num_samples=num_samples)
mcmc.run(
    rng_key, y_obs=y_obs
)
mcmc.print_summary()
samples = mcmc.get_samples()
```

```
sample: 100%|██████████| 3000/3000 [00:01<00:00, 2911.76it/s, 3 steps of size 3.43e-0
1. acc. prob=0.94]
```

	mean	std	median	5.0%	95.0%	n_eff	r_hat
b	-0.15	0.13	-0.15	-0.37	0.05	569.90	1.00
m	1.26	0.10	1.25	1.08	1.42	594.25	1.00

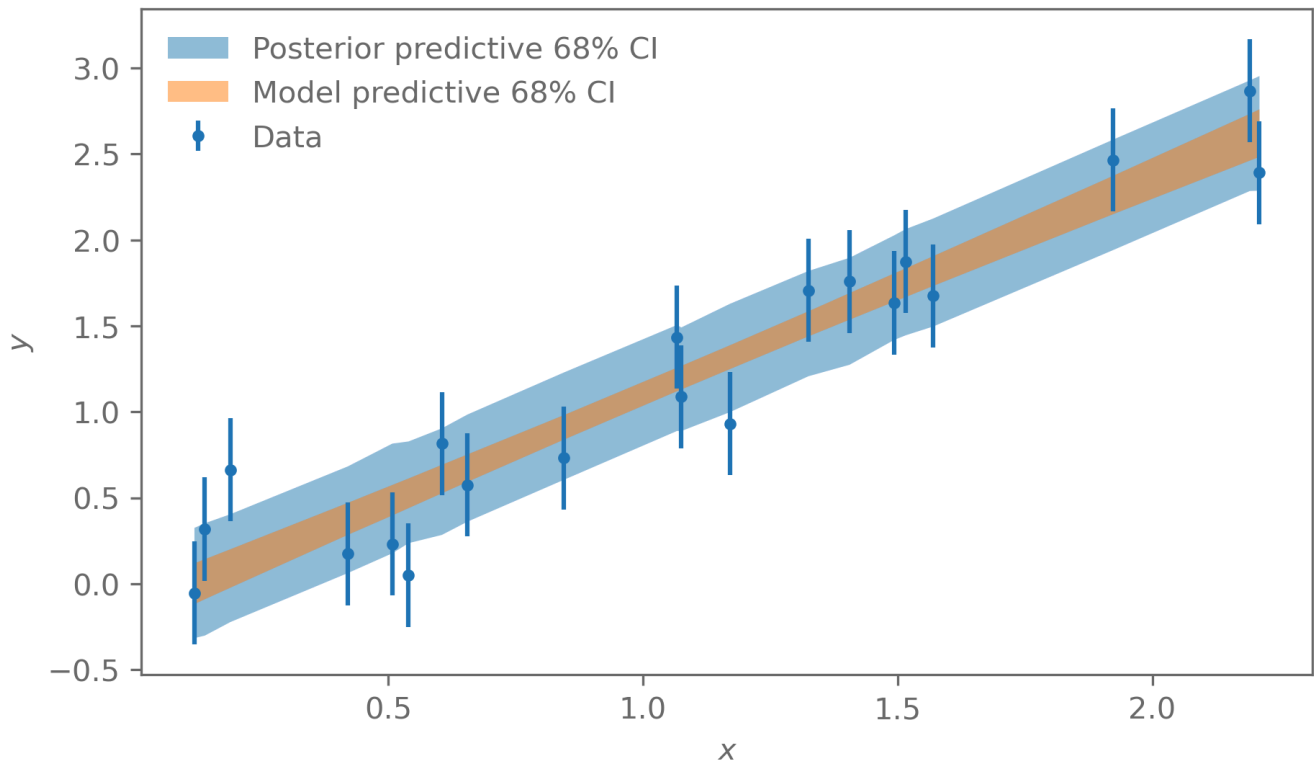
Number of divergences: 0

From here we can look at the posterior samples, or get the posterior predictive distribution:

```
rng_key, master_key = jax.random.split(master_key)

posterior_predictive = numpyro.infer.Predictive(model=model, posterior_samples=samples)
posterior_predictive_samples = posterior_predictive(rng_key=rng_key) ["obs"]

model_predictive_samples = samples["mu"]
```



Now that we have seen how to use `numpyro` on this simple example, we can use it to sample the parameters of the Gaussian process:

```
x_obs, y_obs, y_err = generate_data()
```

```
def model(y_obs=None):
    # We need to define priors for our parameters
    period = numpyro.sample("period", distr.Uniform(low=0.1, high=8.0))
    gamma = numpyro.sample("gamma", distr.HalfNormal(scale=10))
    sigma = numpyro.sample("sigma", distr.HalfNormal(scale=10))
    slope = numpyro.sample("slope", distr.Uniform(low=-1, high=1))
    mean = numpyro.sample("mean", distr.Normal(loc=0, scale=1))

    kernel = sigma**2 * tinygp.kernels.ExpSineSquared(scale=period, gamma=gamma)
    mean_func = lambda x: mean + slope*x
```

```

gp = tinygp.GaussianProcess(kernel, x_obs, diag=sigma_y**2, mean=mean_func)
# The likelihood is Gaussian, so we can use the distribution of the GP
# directly
numpyro.sample("obs", gp.numpyro_dist(), obs=y_obs)

if y_obs is not None:
    conditioned_gp = gp.condition(y=y_obs, X_test=x_pred).gp
    numpyro.deterministic("pred_mean", conditioned_gp.loc)

```

```

# Sample over the model
nuts_kernel = numpyro.infer.NUTS(model, target_accept_prob=0.9)
mcmc = numpyro.infer.MCMC(
    nuts_kernel,
    num_warmup=1000,
    num_samples=1000,
    progress_bar=True,
)
rng_key = jax.random.PRNGKey(55873)
mcmc.run(rng_key, y_obs=y_obs)
samples = mcmc.get_samples()
mcmc.print_summary()

```

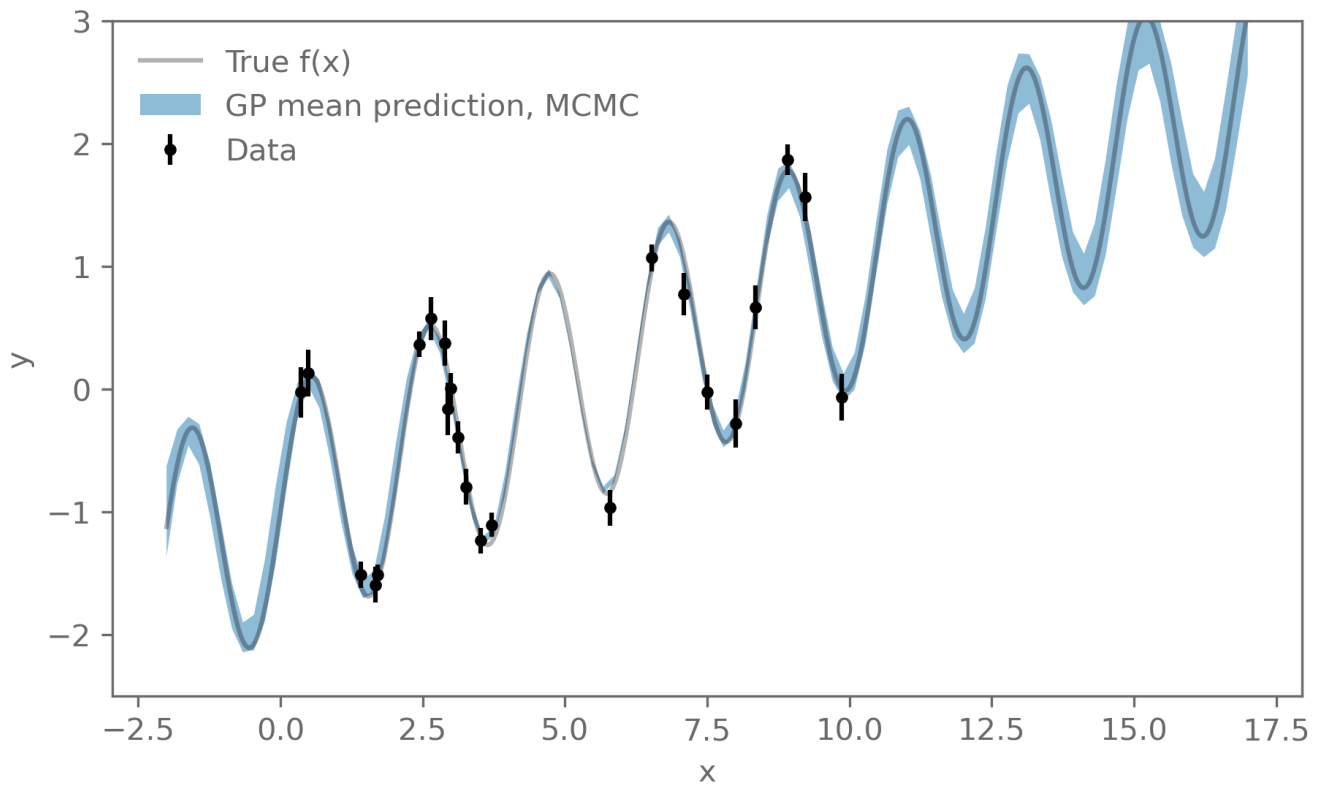
```

sample: 100%|██████████| 2000/2000 [00:02<00:00, 899.05it/s, 7 steps of size 3.25e-0
1. acc. prob=0.92]

```

	mean	std	median	5.0%	95.0%	n_eff	r_hat
gamma	0.67	0.88	0.35	0.00	1.69	659.58	1.00
mean	-0.34	0.83	-0.39	-1.63	1.03	746.80	1.00
period	2.10	0.03	2.10	2.06	2.15	658.48	1.00
sigma	2.43	2.20	1.76	0.42	4.76	406.14	1.00
slope	0.20	0.02	0.20	0.16	0.23	644.55	1.00

Number of divergences: 0



This now accounts for the uncertainty in the kernel (and mean) parameters.

Beside getting this extra uncertainty quantification, this setup also allows us to use a different likelihood.

For example, our data could be photon counts, in which case a Poisson likelihood would be appropriate.

In this example, the Gaussian process model could predict the rate of the Poisson distribution.

```
def model(y_obs=None):
    # ...
    # same setup as before

    gp = tinygp.GaussianProcess(kernel, x_obs, diag=1e-3, mean=mean_func)
    log_rate = numpyro.sample("log_rate", gp.numpyro_dist())

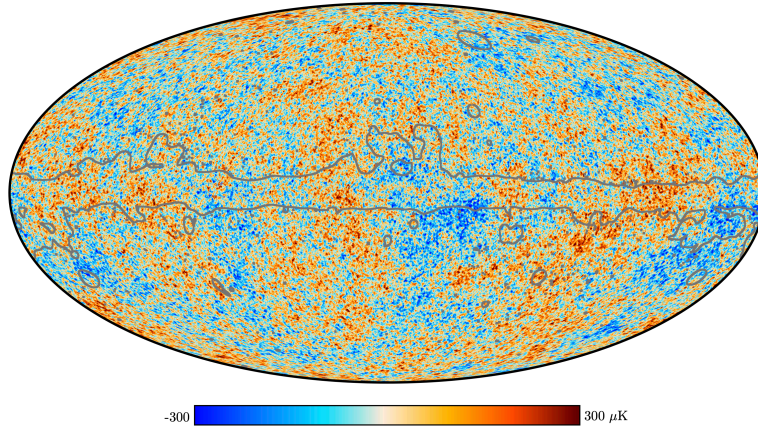
    # We now use a Poisson distribution for the likelihood
    numpyro.sample("obs", distr.Poisson(rate=jnp.exp(log_rate)), obs=y_obs)
```

Note that for a non-Gaussian likelihood, the posterior over the functions is not Gaussian any more, and the exact conditioning does not apply.

Gaussian random fields

The Gaussian process machinery is useful when the observed points are sparse and we want a model to interpolate between the observed data points.

If the process is sampled densely, we might not care about interpolation (because there is nowhere to interpolate).



We still care about the statistical properties of the data, however. Because it is a Gaussian process, if we can describe its covariance and mean, we have extracted all useful information from the data.

So the focus is on finding a good model for the covariance.

In the following we will have a brief look at Gaussian random fields. Gaussian processes and Gaussian random fields are fundamentally the same thing, just different nomenclature in different contexts.

If the Gaussian random field is sampled densely, it is useful to work in harmonic space.

Working with Gaussian random fields in harmonic space has one big advantage: the harmonic coefficients of the field are all distributed as *independent* Gaussian RVs.

If the field $f(\vec{x})$ is defined on $\mathbb{R}^n$, then the harmonic transform is the Fourier transform:

$$\hat{f}(\vec{k}) = \int_{\mathbb{R}^n} d\vec{x} f(\vec{x}) e^{-i\vec{x}\vec{k}}.$$

If $f(\vec{x})$ is a Gaussian random field, its Fourier transform $\hat{f}(\vec{k})$ is drawn from independent Gaussians for each mode $\vec{k}$:

$$\begin{aligned} \text{Re}[\hat{f}(\vec{k})] &\sim \mathcal{N}(0, \frac{P(\vec{k})}{2}) \\ \text{Im}[\hat{f}(\vec{k})] &\sim \mathcal{N}(0, \frac{P(\vec{k})}{2}). \end{aligned}$$

The variance is given in terms of the power spectrum, which is defined as

$$\mathbb{E}[\hat{f}(\vec{k})\hat{f}^*(\vec{k}')] = (2\pi)^n \delta(\vec{k} - \vec{k}') P(\vec{k}).$$

The power spectrum is related to the covariance. Assuming the mean of the field f is zero, we can write the covariance as

$$\begin{aligned} \text{Cov}[f(\vec{x})f(\vec{x} + \vec{r})] &= \mathbb{E}[f(\vec{x})f(\vec{x} + \vec{r})] = \int \frac{d^n k d^n k'}{(2\pi)^{2n}} e^{i\vec{x}\vec{k}} e^{-i(\vec{x}+\vec{r})\vec{k}'} \mathbb{E}[\hat{f}(\vec{k})\hat{f}^*(\vec{k}')] \\ &= \int \frac{d^n k}{(2\pi)^n} e^{-i\vec{r}\vec{k}} P(\vec{k}). \end{aligned}$$

So the power spectrum is the Fourier transform of the covariance.

This connects back to the Gaussian process approach we saw before: if we have a (physical) model for the power spectrum, we can transform that into a model for the covariance/kernel.

The independence of the modes in harmonic space makes building models for the power spectrum sometimes easier.

```
def draw_harmonic_coefficients(power_spectrum_func, size, n_pixel):
    k_abs_grid = get_k_values(size, n_pixel)

    fft_normalization = size**2/n_pixel**4
    power_spectrum_grid = power_spectrum_func(k_abs_grid) / fft_normalization

    coeff_real = np.random.normal(
        loc=0, scale=np.sqrt(power_spectrum_grid/2),
        size=k_abs_grid.shape)
    coeff_imag = np.random.normal(
        loc=0, scale=np.sqrt(power_spectrum_grid/2),
        size=k_abs_grid.shape)

    return coeff_real, coeff_imag

def get_k_values(size, n_pixel):
    k_x = np.fft.fftfreq(n_pixel) * 2*np.pi/size*n_pixel
    k_y = np.fft.rfftfreq(n_pixel) * 2*np.pi/size*n_pixel
    x_idx, y_idx = np.meshgrid(k_x, k_y, indexing="ij")
    k_abs_grid = np.sqrt((x_idx)**2 + (y_idx)**2)

    return k_abs_grid
```

```
coeff_real, coeff_imag = draw_harmonic_coefficients(
    power_spectrum_func, size, n_pixel)

m = np.fft.irfft2(coeff_real + 1j*coeff_imag)
```

```
def power_law_powerspectrum(k, beta):
    return np.pieceswise(k, [k > 0], [lambda k: k**beta, 0])
```

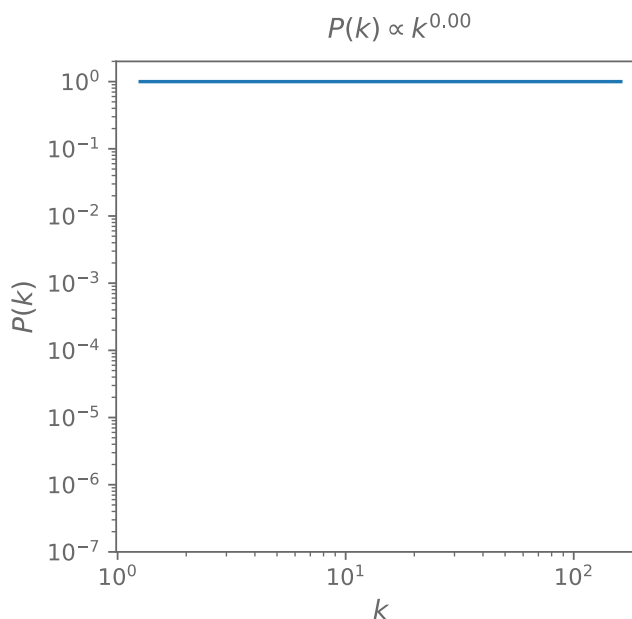
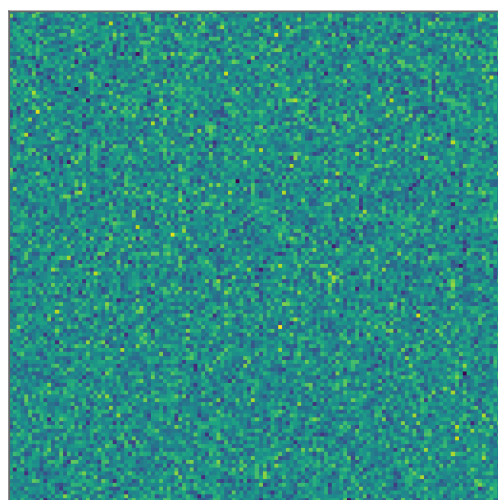
```
def estimate_power_spectrum(field, size, k_bin_edges):
    n_pixel = field.shape[0]
    k_abs_grid = get_k_values(size, n_pixel=n_pixel)

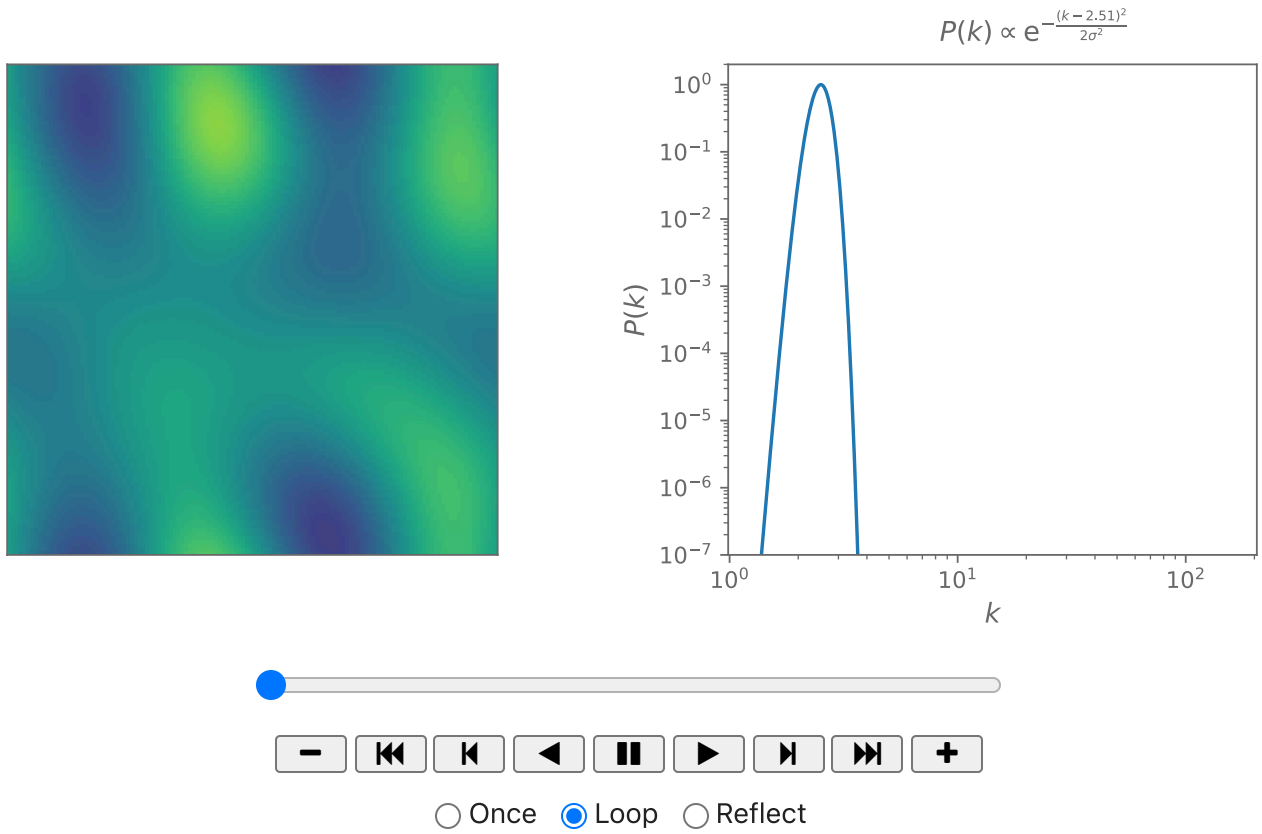
    field_ft = np.fft.rfft2(field)
    field_ft_sq = field_ft * field_ft.conj()

    power_spectrum_estimate = np.zeros(k_bin_edges.size - 1)
    effective_k = np.zeros_like(power_spectrum_estimate)

    for i, (low_bin_edge, high_bin_edge) in enumerate(zip(k_bin_edges[:-1], k_bin_edges[1:])):
        selection = (low_bin_edge <= k_abs_grid) & (k_abs_grid < high_bin_edge)
        power_spectrum_estimate[i] = np.mean(field_ft_sq[selection].real)
        effective_k[i] = np.mean(k_abs_grid[selection])

    fft_normalization = size**2/n_pixel**4
    return power_spectrum_estimate * fft_normalization, effective_k
```





If the domain on which the field is defined is not $\mathbb{R}^n$, then the Fourier transform is not the appropriate transform anymore.

For example, for (isotropic) fields on the sphere, the spherical harmonic transform needs to be used:

$$a_{lm} = \int d\Omega Y_{lm}(\theta, \phi) f(\theta, \phi).$$

The angular power spectrum is defined as

$$\mathbb{E}[a_{lm} a_{l'm'}^*] = \delta_{ll'} \delta_{mm'} C_l.$$

If the field is only defined on an irregular domain, for example because there are sections where there is no data, then some care must be taken.

For a general domain, the harmonic basis that gives independent modes of the Gaussian random field is given by the eigenfunctions of the Laplacian on that domain.

In practice it is very cumbersome to get such an harmonic basis.

It is usually more practicable to keep using the Fourier or spherical harmonic transform and give up on the independence between the harmonic modes of the Gaussian random field.

In this case we say that the field $f(\vec{x})$ is defined everywhere on the domain but what we can observe is the product $W(\vec{x})f(\vec{x})$, where $W(\vec{x})$ is a mask that is one where we have data and zero

elsewhere:

$$W(\vec{x}) = \begin{cases} 1 & \text{inside the region with data} \\ 0 & \text{outside} \end{cases}.$$

The product $W(\vec{x})f(\vec{x})$ becomes a convolution in harmonic space, which couples the different modes together. There are methods to correct for this effect, however. Efficient examples are the SPICE and MASTER algorithms, for example implemented in the `namaster` package ([paper](#)).

The power spectrum of $W(\vec{x})f(\vec{x})$ is called the *pseudo power spectrum*, as its not the power spectrum of the field $f(\vec{x})$ we care about but a convolution with the observational mask.